

shogiAI 資料庫系統報告

資料庫設計方法、互動流程與棋譜資料管理功能

shogiAI 專案

2026 年 5 月

摘要

本報告說明 shogiAI 專案的資料庫系統設計。專案資料來源為 CSA 將棋棋譜，系統將原始棋譜解析為使用者、棋手、對局、走法與局面五類資料，再以 MySQL 儲存。資料庫不只保存原始棋譜內容，也支援棋譜查詢、局面回放、訓練資料匯出、開局庫統計、AI 對弈與模型比賽等功能。設計核心是把「一盤棋」拆成可正規化、可查詢、可重播、可訓練的資料單位，使前端、後端 API 與 AI 模組都能使用同一份可信資料來源。

目录

1	系統目標與資料流程	2
2	資料庫設計方法與原理	2
2.1	設計方法	2
2.2	為什麼要正規化	2
3	資料表設計	3
3.1	users	3
3.2	players	3
3.3	game_records	4
3.4	moves	4
3.5	positions	4
4	關聯設計與資料一致性	5
5	資料匯入流程	5

目錄	2
6 去重與可追溯設計	6
6.1 檔名去重	6
6.2 內容去重	6
6.3 資料追溯	6
7 使用者如何與資料庫互動	6
7.1 棋譜瀏覽	7
7.2 棋譜查詢與篩選	7
7.3 資料庫統計	7
8 資料庫相關功能	7
8.1 棋譜匯入	7
8.2 局面回放	7
8.3 訓練資料匯出	8
8.4 開局庫	8
8.5 AI 對弈與模型比賽	8
9 API 資料結構設計	8
10 索引與效能設計	9
11 安全性與資料品質	9
11.1 SQL 注入防護	9
11.2 路徑安全	9
11.3 合法性檢查	10
12 教授可能提問與回答	10
12.1 為什麼要存 positions，只存 moves 不行嗎	10
12.2 為什麼 moves 和 positions 分開	10
12.3 為什麼用 MySQL 而不是只用檔案	10
12.4 如果棋手同名怎麼辦	10
12.5 資料庫如何支援 AI	10
13 結論	11
A 常用指令	11
A.1 建立資料表並匯入	11
A.2 匯出訓練資料	11
A.3 啟動資料庫瀏覽與 AI API	11

1 系統目標與資料流程

shogiAI 的資料庫系統目標不是單純把 CSA 檔案存進資料庫，而是把棋譜轉換成可被系統操作的資料模型。整體流程如下：

CSA 棋譜 → Python 解析與合法性檢查
→ MySQL 正規化儲存 → HTTP JSON API
→ 前端棋盤、搜尋、AI 與訓練模組

這個流程讓專案具備三個特性：

1. **可追溯**：每一筆訓練樣本都能追回來源對局、手數、SFEN 與實戰走法。
2. **可重播**：系統保存每一步前後的局面，使用者可以直接跳到任意手數。
3. **可擴充**：前端查詢、開局庫、AI 對弈與資料科學訓練都從同一組資料表取得資料。

2 資料庫設計方法與原理

2.1 設計方法

本專案採用關聯式資料庫設計。設計時先分析 CSA 棋譜中有哪些實體，再判斷實體之間的關係：

- 一個系統使用者可以匯入多盤棋。
- 一位棋手可以出現在多盤棋中。
- 一盤棋包含多個走法。
- 一盤棋包含多個局面，其中第 0 手代表初始局面。

因此資料庫拆成五張主要資料表：`users`、`players`、`game_records`、`moves`、`positions`。這種拆法符合第三正規化精神：棋手名稱不重複存在每一手資料中，對局 `metadata` 不混入走法資料，局面資訊也不與走法欄位混在一起。

2.2 為什麼要正規化

若只用一張大表保存所有資料，會產生三個問題：

1. **資料重複**：同一位棋手在多盤棋中重複出現，修改名稱會很困難。
2. **查詢不清楚**：對局資料、走法資料、局面資料混在一起，查詢某一盤棋或某一步時需要額外解析。

3. 訓練資料難匯出：AI 訓練需要的是「局面加下一手」樣本，如果沒有獨立的 positions 與 moves，每次都必須重新讀 CSA 檔案。

正規化後，資料庫能以 SQL 直接取得訓練樣本：

```
SELECT p.sfen, m.usi_move, g.result
FROM positions p
JOIN moves m
  ON m.game_id = p.game_id
  AND m.move_number = p.move_number + 1
JOIN game_records g
  ON g.game_id = p.game_id
ORDER BY p.game_id, p.move_number;
```

3 資料表設計

3.1 users

users 保存系統使用者或匯入者。即使目前主要由匯入程式使用，保留使用者表可以讓日後擴充登入、權限、個人收藏或上傳紀錄。

欄位	型別	用途
user_id	INT PK	使用者主鍵，自動遞增。
username	VARCHAR	使用者名稱或匯入程式名稱。
email	VARCHAR	可選的唯一信箱。
password_hash	VARCHAR	預留登入驗證欄位。
role	VARCHAR	權限角色，例如 user。
created_at	DATETIME	建立時間。

3.2 players

players 保存棋手資料，避免同一棋手名稱在每盤棋中重複儲存。

欄位	型別	用途
player_id	INT PK	棋手主鍵。
player_name	VARCHAR	棋手名稱。
rank_name	VARCHAR	段位或稱號，可為空。
created_at	DATETIME	建立時間。

3.3 game_records

game_records 是對局主表，保存一盤棋的 metadata 與外鍵關係。

欄位	型別	用途
game_id	INT PK	對局主鍵。
uploader_id	INT FK	匯入者，連到 users。
black_player_id	INT FK	先手棋手，連到 players。
white_player_id	INT FK	後手棋手，連到 players。
event_name	VARCHAR	賽事名稱。
site	VARCHAR	地點。
opening	VARCHAR	開局名稱。
result	VARCHAR	結果，例如 TORYO、DRAW、SENNICHITE。
source_format	VARCHAR	來源格式，目前主要為 CSA。
original_file_name	VARCHAR	原始檔名，用於去重與追溯。
played_at	DATE	對局日期。

3.4 moves

moves 保存每一步棋。系統同時保存 original_move 與 usi_move，目前兩者多數相同，但保留原始欄位可支援未來不同格式匯入。

欄位	型別	用途
move_id	INT PK	走法主鍵。
game_id	INT FK	所屬對局。
move_number	INT	第幾手，從 1 開始。
side_to_move	ENUM	下這步棋的一方：black 或 white。
original_move	VARCHAR	原始走法字串。
usi_move	VARCHAR	標準化 USI 走法。
move_time	VARCHAR	該手耗時。
comment	TEXT	棋譜註解。

3.5 positions

positions 保存每一手後的棋盤狀態。第 0 筆局面代表初始盤面，第 n 筆局面代表走完第 n 手後的狀態。

欄位	型別	用途
position_id	INT PK	局面主鍵。
game_id	INT FK	所屬對局。
move_number	INT	此局面對應到走完第幾手。
side_to_move	ENUM	輪到哪一方行棋。
sfen	TEXT	局面的 SFEN 表示。
sfen_hash	CHAR(64)	SFEN 的 SHA-256 hash，便於比對重複局面。
is_check	BOOLEAN	是否正在被將軍。
legal_moves_count	INT	合法手數量。

4 關聯設計與資料一致性

主要關係如下：

$$\text{users} \xrightarrow{1:N} \text{game_records} \xleftarrow{N:1} \text{players}$$

$$\text{game_records} \xrightarrow{1:N} \text{moves} \quad \text{game_records} \xrightarrow{1:N} \text{positions}$$

其中 moves 與 positions 的對應關係是本系統最重要的設計：

$$Position_t + Move_{t+1} \rightarrow Position_{t+1}$$

這表示第 t 手後的局面，加上下一手 $Move_{t+1}$ ，會得到第 $t+1$ 手後的局面。這個設計同時服務三種需求：

- 前端可以直接查 positions 顯示任意手數，不需要每次從第一手重播。
- AI 訓練可以用 $Position_t$ 當輸入， $Move_{t+1}$ 當 policy 標籤。
- 系統可以用 sfen_hash 判斷重複局面或建立開局統計。

5 資料匯入流程

資料匯入由 src/import_csa_to_db.py 負責。核心流程如下：

1. 掃描單一 CSA 檔案或整個資料夾。
2. 使用 cshogi.CSA.Parser 解析棋譜。

3. 建立或取得匯入者與棋手資料。
4. 建立 `game_records`。
5. 從初始 SFEN 建立 `cshogi.Board`。
6. 逐手檢查 `board.is_legal(move)`。
7. 寫入 `moves`，再推進棋盤並寫入 `positions`。
8. 每盤棋獨立 `commit`；若有錯誤則 `rollback` 並記錄。

匯入程式也提供 `--dry-run`、`--skip-existing`、`--max-games`、`--pause-every` 等參數。這些參數使系統能在學校 MySQL 帳號有查詢次數限制時分批匯入，並避免重複資料。

6 去重與可追溯設計

6.1 檔名去重

`original_file_name` 可用來判斷某個 CSA 檔案是否已匯入。如果使用 `--skip-existing`，系統會略過已存在的檔名。

6.2 內容去重

只用檔名去重不夠，因為同一盤棋可能被重新命名。因此系統另外建立內容 key：

$$\text{content key} = \text{SHA256}(\text{initial SFEN} + \text{full USI move sequence})$$

只要初始局面與完整走法序列相同，就視為同一盤棋。這能避免同一盤棋因檔名不同而重複匯入。

6.3 資料追溯

每筆訓練資料都能從 `game_id`、`move_number`、`sfen` 與 `usi_move` 追溯回原始對局。教授若問「模型某個樣本從哪裡來」，可回答：由 `positions` 與 `moves` join 得到，並可透過 `game_records.original_file_name` 找回原始 CSA。

7 使用者如何與資料庫互動

使用者不直接寫 SQL，而是透過後端 API 與前端操作間接使用資料庫。後端程式是 `src/csa_browser_api.py`，前端是 `web/index.html` 與 `web/app.js`。

7.1 棋譜瀏覽

使用者可以載入對局列表、選擇對局、前後移動手數、跳到第一手或最後一手。前端會呼叫：

```
GET /api/games
GET /api/games/<game_id>?ply=<n>
```

後端從 `game_records`、`moves`、`positions` 取資料，再序列化為 JSON。

7.2 棋譜查詢與篩選

前端提供賽事、日期、棋手、開局等篩選條件。對應欄位來自：

- 賽事：`game_records.event_name`
- 日期：`game_records.played_at`
- 棋手：`players.player_name`
- 開局：`game_records.opening`

7.3 資料庫統計

前端可顯示資料庫目前有多少對局、棋手、走法、局面與重複局面群組。後端 API 為：

```
GET /api/db/stats
```

這讓使用者知道資料庫規模，也方便檢查匯入是否成功。

8 資料庫相關功能

8.1 棋譜匯入

匯入功能支援單一檔案、整個資料夾與遞迴掃描。若教授問「系統如何避免非法棋譜污染資料庫」，答案是：匯入時每一步都經過 `cshogi.Board.is_legal` 驗證，遇到錯誤會 `rollback` 並記錄錯誤，不會把半盤棋寫進資料庫。

8.2 局面回放

系統將每一手後的 SFEN 保存到 `positions`，因此前端可以任意跳轉，不必每次從初始局面重算。這是資料庫換取查詢速度的設計：多存一些局面資料，降低使用者互動時的延遲。

8.3 訓練資料匯出

`src/export_policy_dataset.py` 從 MySQL 匯出 `out/policy_dataset.npz`。它把每筆 `position` 轉成神經網路輸入，把下一手 `move` 轉成 `policy label`，並根據對局結果產生 `value label`。

8.4 開局庫

`OpeningBook.from_mysql` 會統計資料庫中前若干手的局面與下一手頻率。AI 對弈時，如果目前局面存在於開局庫，系統可優先選擇資料庫中常見走法。這是一種資料庫驅動的 AI 功能。

8.5 AI 對弈與模型比賽

使用者可以透過前端與 AI 對弈，或讓兩個模型互相對弈。AI 搜尋會使用資料庫訓練出的 `policy/value model`，也可以使用資料庫建立的 `opening book`。模型比賽結果則透過 API 回傳給前端顯示。

9 API 資料結構設計

後端不直接把資料庫 `row` 原樣送到前端，而是轉成適合 UI 使用的 JSON。例如局面 API 會回傳：

```
{
  "game": {...},
  "ply": 30,
  "maxPly": 105,
  "turn": "+",
  "board": [...],
  "hands": {"+": {...}, "-": {...}},
  "lastMove": {...},
  "moves": [...],
  "isCheck": false,
  "legalMovesCount": 42
}
```

這個設計把「資料庫儲存格式」與「前端顯示格式」分開。資料庫關心正規化與查詢效率，前端關心棋盤格子、持駒、最後一步與可視化資訊。

10 索引與效能設計

本專案適合加入下列索引：

```
CREATE INDEX idx_game_records_played_at
ON game_records(played_at);

CREATE INDEX idx_game_records_original_file_name
ON game_records(original_file_name);

CREATE INDEX idx_moves_game_number
ON moves(game_id, move_number);

CREATE INDEX idx_positions_game_number
ON positions(game_id, move_number);

CREATE INDEX idx_positions_sfen_hash
ON positions(sfen_hash);
```

其中 `idx_positions_game_number` 最重要，因為前端查某盤棋第 n 手局面時會頻繁使用 `game_id` 與 `move_number`。`idx_moves_game_number` 則支援訓練資料匯出與棋譜回放。

11 安全性與資料品質

11.1 SQL 注入防護

匯入與查詢使用 PyMySQL 參數化查詢，例如：

```
cursor.execute(
    "SELECT user_id FROM users WHERE email = %s",
    (email,)
)
```

使用參數化查詢可以避免使用者輸入被拼接成 SQL 指令。

11.2 路徑安全

若系統從本機 CSA 檔案讀資料，會檢查解析後路徑仍在 `data/` 目錄內，避免使用者透過路徑穿越讀取專案外檔案。

11.3 合法性檢查

資料品質的核心是 `cslogi`。每一步棋在寫入資料庫前都必須合法，否則視為錯誤棋譜。這能確保後續 AI 訓練不會學到不合法走法。

12 教授可能提問與回答

12.1 為什麼要存 `positions`，只存 `moves` 不行嗎

只存 `moves` 的確可以重建棋局，但每次查第 100 手都要從第 0 手重播 100 次。保存 `positions` 可以讓前端直接查詢任意局面，也讓 AI 訓練能快速取得狀態。這是以儲存空間換查詢速度與系統可用性。

12.2 為什麼 `moves` 和 `positions` 分開

`moves` 表示動作，`positions` 表示狀態。兩者概念不同，分開後能清楚描述：

$$S_t + A_{t+1} = S_{t+1}$$

這個設計也剛好符合機器學習資料格式：輸入是狀態 S_t ，標籤是下一步動作 A_{t+1} 。

12.3 為什麼用 MySQL 而不是只用檔案

檔案適合保存原始資料，但不適合多條件查詢、統計、去重、局面索引與多人資料管理。MySQL 提供關聯、外鍵、索引與交易機制，能讓資料更可靠，也讓前端與 AI 模組共用一致資料來源。

12.4 如果棋手同名怎麼辦

目前以 `player_name` 與 `rank_name` 作為棋手識別基礎。若未來需要更精準，可以加入棋手官方 ID、來源網站 ID 或人工合併表。現階段資料來源多為棋譜檔案，姓名加段位已能滿足基本正規化需求。

12.5 資料庫如何支援 AI

資料庫提供兩種 AI 支援：第一，匯出訓練資料，讓模型學習局面到走法與勝負的對應；第二，統計開局庫，讓 AI 在前期可參考資料庫中的常見實戰走法。

13 結論

shogiAI 的資料庫系統將原本分散的 CSA 棋譜轉成關聯式資料模型。五張核心表各自負責使用者、棋手、對局、走法與局面，並透過外鍵與手數建立清楚關係。這個設計讓使用者可以查詢與回放棋譜，讓後端可以提供穩定 JSON API，也讓數據科學模組可以直接匯出可訓練資料。整體而言，資料庫不只是資料存放處，而是連接棋譜瀏覽、AI 訓練、AI 對弈與模型評估的核心基礎。

A 常用指令

A.1 建立資料表並匯入

```
python src\import_csa_to_db.py ^  
  --input data ^  
  --recursive ^  
  --create-tables ^  
  --host 140.135.65.53 ^  
  --port 3306 ^  
  --user 11211213 ^  
  --database DB11211213 ^  
  --skip-existing
```

A.2 匯出訓練資料

```
python src\export_policy_dataset.py ^  
  --output out\policy_dataset.npz ^  
  --host 140.135.65.53 ^  
  --port 3306 ^  
  --user 11211213 ^  
  --database DB11211213
```

A.3 啟動資料庫瀏覽與 AI API

```
python src\csa_browser_api.py ^  
  --source mysql ^  
  --db-host 140.135.65.53 ^  
  --db-port 3306 ^  
  --db-user 11211213 ^
```

```
--db-name DB11211213 ^  
--policy-model out\policy_model.pt
```